

CS 2006 - Operating Systems

Chrono Rift

Semester Project Report

Spring 2026

Members:

Abdul Moeed - 24I-0720

Hamza Sheikh - 24I-0845

Section: CS-A

1. Project Overview

Chrono Rift is a multi-process tactical RPG written in C++ for Linux. The project uses shared memory, unnamed semaphores, POSIX threads, fork, waitpid, and signals.

The game has human-controlled players and computer-controlled enemies. Every entity has HP, damage, speed, stamina, and alive status. Stamina decides whose turn comes next.

The Arbiter is the only process that changes the real game state. HIP and ASP only submit action requests. This keeps the game rules consistent.

2. System Architecture

The project uses three processes. They communicate through one System V shared memory segment. Pipes and FIFOs are not used.

2.1 Game Arbiter

The Arbiter is the parent process. It creates shared memory, initializes the state, forks HIP and ASP, runs the scheduler, applies actions, checks win/loss, and cleans up at the end.

The Arbiter also creates a rendering pthread and a deadlock monitor pthread.

2.2 Human Interfacing Process

HIP manages player input. It creates one pthread for each player character. Only the active player thread wakes up when Arbiter gives that player a turn.

HIP waits for GUI input, writes PendingAction, and posts ActionReady. HIP does not directly change HP, stamina, inventory, or artifacts.

2.3 Automated Strategic Process

ASP manages enemies. It creates one pthread for each enemy. Only the active enemy thread wakes up on its turn.

The enemy sleeps for 0 to 2 seconds to simulate thinking. Then it submits Strike, Skip, or Acquire Artifact to Arbiter.

3. Shared Memory Layout

All shared data is inside SharedState. Arbiter creates it with shmget and attaches with shmat. HIP and ASP attach using the shmid passed through argv.

Players[4]: HP, stamina, speed, damage, alive flag, stun state, artifact state, and ready order.

Enemies[9]: enemy stats, weapon state, death time, artifact state, and ready order.

Inventory[4][20]: primary inventory slots for each player.

Lts[4]: long term storage for swapped out items.

Artifacts[3]: Solar Core, Lunar Blade, and Eclipse Relic ownership table.

PendingAction: the action written by HIP or ASP.

Turn: active entity, player/enemy turn flag, phase, and turn id.

Gui: GUI action choices written by renderer and read by HIP.

DropEvent: pending weapon drop information.

Log[50]: action log ring buffer.

Semaphores: GlobalLock, PlayerTurnReady, EnemyTurnReady, ActionReady, and InputLock.

GlobalLock works like a binary mutex. It protects shared state from race conditions.

4. Temporal Scheduling and Stamina Logic

Every second Arbiter adds speed to stamina for every alive and non-stunned entity. Stamina is capped at max stamina.

When an entity reaches full stamina, Arbiter increments SchedulerCounter and stores that value in the entity ReadyOrder field.

The next turn goes to the full-stamina entity with the smallest ReadyOrder. This means the entity that became ready first acts first.

Each turn also has a TurnId. HIP and ASP must submit the same TurnId. If an old action arrives late, Arbiter rejects it.

After a normal action, stamina becomes 0. After Skip or timeout, stamina becomes 50 percent. ReadyOrder is reset after the turn.

5. Signal Mechanics

5.1 Stun

Stun uses SIGUSR1. Arbiter sets Stunned to 1 and sets StunEndTime to current time plus 3 seconds. Then Arbiter sends SIGUSR1 to HIP or ASP.

The signal handler sleeps for 3 seconds. Arbiter also checks StunEndTime in the scheduler. A stunned entity cannot act until the stun has ended.

5.2 Ultimate Ability

Ultimate requires Solar Core and Lunar Blade in the player active inventory. Both must also belong to that player in the artifact table.

When Ultimate is used, Arbiter sends SIGSTOP to ASP and starts alarm(10). During this time ASP is frozen and Arbiter does not schedule enemy turns. When SIGALRM arrives, Arbiter sends SIGCONT to ASP.

5.3 Quit

Quit uses SIGTERM. HIP sends SIGTERM to Arbiter. Arbiter sets GameOver, wakes blocked threads, waits for child processes, destroys semaphores, detaches shared memory, and removes it with IPC_RMID.

6. Inventory Space Allocator

Each player has 20 primary inventory slots. Every weapon needs contiguous slots.

allocate_weapon uses first fit. It scans from slot 0 and places the item in the first free block that is large enough.

If no block is large enough, auto_swap_out_for_space checks possible inventory windows and swaps out the fewest weapons needed to create space. The swapped items go to long term storage.

Swap In brings an item back from long term storage. It costs a full turn. If needed, the same swap-out rule is used.

Solar Core and Lunar Blade each take 10 slots. Holding both fills all 20 slots. The allocator protects this case so one artifact is not swapped out while trying to hold the other.

7. Artifact System and Deadlock Detection

The artifacts are Solar Core, Lunar Blade, and Eclipse Relic. The shared artifact table stores whether each artifact is free, who holds it, and whether the holder is a player or enemy.

Each entity has ArtifactHeld[3]. This lets one entity hold Solar Core and Lunar Blade at the same time.

Solar Core and Lunar Blade also take inventory space because the project weapon table gives them slot size and damage. They can be used as weapons only by the player that owns them. Eclipse Relic is artifact-only because no slot size or damage is given for it.

If an artifact is already held, the requesting entity records what it is waiting for. The deadlock monitor thread checks these waits every 2 seconds.

The monitor builds a wait-for graph. If it finds a cycle, it forces one entity to release an artifact. This breaks the deadlock and lets the game continue.

8. SFML Rendering Thread

The rendering thread is a pthread inside Arbiter. It owns the SFML window.

Each frame it locks GlobalLock, copies SharedState to a local snapshot, unlocks, and draws from the snapshot. This keeps the lock time short.

The display shows HP bars, stamina bars, inventory slots, artifact state, action log, and the player action menu.

9. Weapon Drop System

When an enemy dies without already holding a weapon, Arbiter rolls a 50 percent drop chance. Solar Core and Lunar Blade are not random drops because they are unique artifacts.

If the player accepts the drop, Arbiter places it in inventory using the allocator. If the player rejects it, an alive enemy picks it up.

10. Eclipse Relic

Eclipse Relic starts outside the world. After at least 3 enemies are killed, Arbiter rolls a 30 percent chance to introduce it.

Once introduced, it follows the same locking and deadlock rules as the other artifacts.

11. Roll Number Configuration

My roll number is 0720. The game uses 720 as the random seed.

Player HP = 720 plus random 100 to 1000.

Enemy HP = 20 plus random 50 to 200.

Player damage = 0 plus 10 = 10.

Enemy damage = 2 plus 10 = 12.

Player speed = 100 divided by party size.

Enemy speed = random 10 to 30.

Player max stamina = 100.

Enemy max stamina = 150.

11.1 Weapon Table

Weapon Name	Slot Size	Damage Output
Solar Core	10	95
Lunar Blade	10	90
Iron Halberd	7	55
Venom Dagger	4	30
Thunderstaff	6	50
Obsidian Axe	5	45
Frostbow	6	48
Splinter Stick	2	12

12. Actions

Player actions are Strike, Exhaust, Use Weapon, Swap In, Heal, Skip, Ultimate, Acquire Artifact, and Quit.

Enemy actions are Strike, Skip, and Acquire Artifact.

Strike reduces HP. Exhaust reduces enemy stamina. Use Weapon applies weapon damage. Heal restores 10 percent HP. Swap In costs the full turn. Skip sets stamina to half. Ultimate freezes ASP for 10 seconds.

13. Win, Lose, and Quit

The players win after killing 10 enemies. The players lose if all player characters die. The game quits when the player selects Quit.

In all cases Arbiter sets GameOver and performs clean shutdown.

14. Synchronization Summary

GlobalLock protects shared state.

PlayerTurnReady wakes HIP for player turns.

EnemyTurnReady wakes ASP for enemy turns.

ActionReady tells Arbiter that an action was submitted.

InputLock serializes GUI input.

Per-player semaphores wake the correct player thread.

Per-enemy semaphores wake the correct enemy thread.

TurnId prevents late actions from being applied to the wrong turn.

15. Turnaround Time Analysis

Turnaround time means the time from stamina starting at 0 until the entity gets its first turn.

Formula: Turn time in seconds = ceiling(Max Stamina / Speed)

Ceiling is used because stamina increases once per second.

15.1 Player Turnaround Time

Party Size	Player Speed	Max Stamina	First Turn Time
1 Player	100	100	1 second
2 Players	50	100	2 seconds
3 Players	33	100	4 seconds
4 Players	25	100	4 seconds

For 3 players, stamina is 99 after 3 seconds. It reaches 100 on the 4th second.

15.2 Enemy Turnaround Time

Enemy Speed	Max Stamina	Formula	First Turn Time
10	150	$150 / 10$	15 seconds
15	150	$150 / 15$	10 seconds
20	150	$150 / 20$	8 seconds
25	150	$150 / 25$	6 seconds
30	150	$150 / 30$	5 seconds

Enemy speed is random from 10 to 30. So enemies can get their first turn between 5 and 15 seconds.

15.3 Ready Order

If more than one entity is full, Arbiter uses ReadyOrder. The entity with the smallest ReadyOrder acts first. This means the entity that reached full stamina first gets the turn first.

After the turn, ReadyOrder is reset. This makes later turns follow the same stamina formula again.

16. Gameplay Screenshots

17. Conclusion

Chrono Rift uses shared memory for communication, semaphores for synchronization, pthreads for independent entities, and signals for stun, ultimate pause, and quit.

The Arbiter-only rule keeps game logic clean. HIP and ASP only submit actions, so the shared state stays consistent.

The project compiles and runs inside Ubuntu with SFML and the provided Makefile.